



8.5 Hands-on Lab — Secure & Deploy AI Workloads with GPU-Enabled Kubernetes on Azure



Objective

Set up a **GPU-enabled AKS (Azure Kubernetes Service) cluster**, integrate NVIDIA GPU Operator, deploy an AI workload securely, and apply **enterprise-grade security controls**.



Prerequisites

- Azure subscription with sufficient quota for GPU VMs (e.g., NC, ND, or NV-series)
 - Azure CLI installed (`az`)
 - kubectl & Helm installed locally
 - Access to **NVIDIA NGC account** (for containers and models)
 - AI model or container image ready for deployment
 - Basic familiarity with Kubernetes security concepts
-

Step 1 — Create a Resource Group

```
az group create \  
  --name ai-secure-rg \  
  --location eastus
```

Purpose: Keeps all related resources grouped for easy cleanup and management.

Step 2 — Deploy GPU-Enabled AKS Cluster

```
az aks create \  
  --resource-group ai-secure-rg \  
  --name ai-secure-aks \  
  --node-count 2 \  
  --node-vm-size Standard_NC6s_v3 \  
  --enable-managed-identity \  
  --enable-aad \  
  --network-plugin azure \  
  --generate-ssh-keys
```

Notes:

- `Standard_NC6s_v3` includes NVIDIA Tesla V100 GPU.

- `--enable-aad` enforces Azure AD authentication for cluster access.

Step 3 — Connect to the Cluster

```
az aks get-credentials \  
  --resource-group ai-secure-rg \  
  --name ai-secure-aks
```

Validation:

```
kubectl get nodes -o wide
```

You should see GPU-enabled nodes.

Step 4 — Install NVIDIA GPU Operator

```
helm repo add nvidia https://nvidia.github.io/gpu-operator  
helm repo update
```

```
helm install gpu-operator nvidia/gpu-operator \  
  --namespace gpu-operator \  
  --create-namespace
```

Purpose: Automates driver, CUDA toolkit, and monitoring stack installation.

Step 5 — Enable GPU Access Validation

```
kubectl run gpu-test --rm -it \  
  --image=nvidia/cuda:11.8.0-base-ubuntu22.04 \  
  --restart=Never -- nvidia-smi
```

If successful, you'll see GPU details from the node.

Step 6 — Create Namespace and Apply RBAC

```
kubectl create namespace ai-secure
```

```
kubectl create role ai-secure-role \  
  --namespace ai-secure \  
  --verb=get,list,watch,create,update,delete \  
  --resource=pods,services,deployments
```

```
kubectl create rolebinding ai-secure-binding \  
  --role=ai-secure-role \  
  --user=<AAD-User-Email> \  
  --namespace=ai-secure
```

```
--namespace=ai-secure
```

Purpose: Restricts workload access to only approved users.

Step 7 — Secure Image Pull from Private Registry

1. Create Azure Container Registry (ACR):

```
az acr create --resource-group ai-secure-rg \  
  --name aiacrsecure \  
  --sku Standard
```

2. Import NVIDIA NGC container into ACR:

```
az acr import --name aiacrsecure \  
  --source nvcr.io/nvidia/tritonserver:23.10-py3
```

3. Create Kubernetes Secret for private registry:

```
kubectl create secret docker-registry acr-secret \  
  --docker-server=aiacrsecure.azurecr.io \  
  --docker-username=<username> \  
  --docker-password=<password> \  
  --namespace ai-secure
```

Step 8 — Deploy AI Workload via Helm

```
helm install triton-inference nvidia/triton-inference-server \  
  --namespace ai-secure \  
  --set image.repository=aiacrsecure.azurecr.io/tritonserver \  
  --set image.tag=23.10-py3 \  
  --set resources.limits.nvidia.com/gpu=1 \  
  --set persistence.enabled=true \  
  --set service.type=LoadBalancer \  
  --set securityContext.readOnlyRootFilesystem=true \  
  --set securityContext.runAsNonRoot=true \  
  --set securityContext.capabilities.drop[0]=ALL
```

Security Features Added:

- Read-only filesystem
 - Run as non-root
 - Drop all unnecessary Linux capabilities
-

Step 9 — Apply Network Policy

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-internal
  namespace: ai-secure
spec:
  podSelector: {}
  ingress:
    - from:
      - podSelector: {}
```

```
kubectl apply -f network-policy.yaml
```

Purpose: Restricts pod communication to internal namespace traffic only.

Step 10 – Enable Monitoring

```
kubectl apply -f https://raw.githubusercontent.com/NVIDIA/dcgm-exporter/main/deploy/kubernetes/dcgm-exporter.yaml
```

Integrates with Prometheus for GPU health metrics.

Step 11 – Test Inference Endpoint

Get external IP:

```
kubectl get svc -n ai-secure
```

Send test request:

```
curl -v http://<external-ip>:8000/v2/health/ready
```

Step 12 – Clean Up (Optional)

```
az group delete --name ai-secure-rg --yes --no-wait
```

✅ Outcome:

You now have a **GPU-enabled Azure Kubernetes cluster** running a **secure NVIDIA Triton Inference Server**, integrated with private registry access, RBAC, network policies, and GPU monitoring.